

```

54     if (allStacksAreFull()) {
55         throw new FullStackException();
56     }
57
58     /* If this stack is full, expand it. */
59     StackInfo stack = info[stackNum];
60     if (stack.isFull()) {
61         expand(stackNum);
62     }
63
64     /* Find the index of the top element in the array + 1, and increment the
65      * stack pointer */
66     stack.size++;
67     values[stack.lastElementIndex()] = value;
68 }
69
70 /* Remove value from stack. */
71 public int pop(int stackNum) throws Exception {
72     StackInfo stack = info[stackNum];
73     if (stack.isEmpty()) {
74         throw new EmptyStackException();
75     }
76
77     /* Remove last element. */
78     int value = values[stack.lastElementIndex()];
79     values[stack.lastElementIndex()] = 0; // Clear item
80     stack.size--; // Shrink size
81     return value;
82 }
83
84 /* Get top element of stack.*/
85 public int peek(int stackNum) {
86     StackInfo stack = info[stackNum];
87     return values[stack.lastElementIndex()];
88 }
89 /* Shift items in stack over by one element. If we have available capacity, then
90  * we'll end up shrinking the stack by one element. If we don't have available
91  * capacity, then we'll need to shift the next stack over too. */
92 private void shift(int stackNum) {
93     System.out.println("/// Shifting " + stackNum);
94     StackInfo stack = info[stackNum];
95
96     /* If this stack is at its full capacity, then you need to move the next
97      * stack over by one element. This stack can now claim the freed index. */
98     if (stack.size >= stack.capacity) {
99         int nextStack = (stackNum + 1) % info.length;
100         shift(nextStack);
101         stack.capacity++; // claim index that next stack lost
102     }
103
104     /* Shift all elements in stack over by one. */
105     int index = stack.lastCapacityIndex();
106     while (stack.isWithinStackCapacity(index)) {
107         values[index] = values[previousIndex(index)];
108         index = previousIndex(index);
109     }

```

```

110
111     /* Adjust stack data. */
112     values[stack.start] = 0; // Clear item
113     stack.start = nextIndex(stack.start); // move start
114     stack.capacity--; // Shrink capacity
115 }
116
117 /* Expand stack by shifting over other stacks */
118 private void expand(int stackNum) {
119     shift((stackNum + 1) % info.length);
120     info[stackNum].capacity++;
121 }
122
123 /* Returns the number of items actually present in stack. */
124 public int numberOfElements() {
125     int size = 0;
126     for (StackInfo sd : info) {
127         size += sd.size;
128     }
129     return size;
130 }
131
132 /* Returns true is all the stacks are full. */
133 public boolean allStacksAreFull() {
134     return numberOfElements() == values.length;
135 }
136
137 /* Adjust index to be within the range of 0 -> length - 1. */
138 private int adjustIndex(int index) {
139     /* Java's mod operator can return neg values. For example, (-11 % 5) will
140      * return -1, not 4. We actually want the value to be 4 (since we're wrapping
141      * around the index). */
142     int max = values.length;
143     return ((index % max) + max) % max;
144 }
145
146 /* Get index after this index, adjusted for wrap around. */
147 private int nextIndex(int index) {
148     return adjustIndex(index + 1);
149 }
150
151 /* Get index before this index, adjusted for wrap around. */
152 private int previousIndex(int index) {
153     return adjustIndex(index - 1);
154 }
155 }

```

In problems like this, it's important to focus on writing clean, maintainable code. You should use additional classes, as we did with `StackInfo`, and pull chunks of code into separate methods. Of course, this advice applies to the "real world" as well.

3.2 Stack Min: How would you design a stack which, in addition to push and pop, has a function `min` which returns the minimum element? Push, pop and `min` should all operate in $O(1)$ time.

pg 98

SOLUTION

The thing with minimums is that they don't change very often. They only change when a smaller element is added.

One solution is to have just a single `int` value, `minValue`, that's a member of the `Stack` class. When `minValue` is popped from the stack, we search through the stack to find the new minimum. Unfortunately, this would break the constraint that push and pop operate in $O(1)$ time.

To further understand this question, let's walk through it with a short example:

```
push(5); // stack is {5}, min is 5
push(6); // stack is {6, 5}, min is 5
push(3); // stack is {3, 6, 5}, min is 3
push(7); // stack is {7, 3, 6, 5}, min is 3
pop();   // pops 7. stack is {3, 6, 5}, min is 3
pop();   // pops 3. stack is {6, 5}. min is 5.
```

Observe how once the stack goes back to a prior state (`{6, 5}`), the minimum also goes back to its prior state (5). This leads us to our second solution.

If we kept track of the minimum at each state, we would be able to easily know the minimum. We can do this by having each node record what the minimum beneath itself is. Then, to find the `min`, you just look at what the top element thinks is the `min`.

When you push an element onto the stack, the element is given the current minimum. It sets its "local `min`" to be the `min`.

```
1 public class StackWithMin extends Stack<NodeWithMin> {
2     public void push(int value) {
3         int newMin = Math.min(value, min());
4         super.push(new NodeWithMin(value, newMin));
5     }
6
7     public int min() {
8         if (this.isEmpty()) {
9             return Integer.MAX_VALUE; // Error value
10        } else {
11            return peek().min;
12        }
13    }
14 }
15
16 class NodeWithMin {
17     public int value;
18     public int min;
19     public NodeWithMin(int v, int min){
20         value = v;
21         this.min = min;
22     }
23 }
```

There's just one issue with this: if we have a large stack, we waste a lot of space by keeping track of the `min` for every single element. Can we do better?

We can (maybe) do a bit better than this by using an additional stack which keeps track of the mins.

```

1  public class StackWithMin2 extends Stack<Integer> {
2      Stack<Integer> s2;
3      public StackWithMin2() {
4          s2 = new Stack<Integer>();
5      }
6
7      public void push(int value){
8          if (value <= min()) {
9              s2.push(value);
10         }
11         super.push(value);
12     }
13
14     public Integer pop() {
15         int value = super.pop();
16         if (value == min()) {
17             s2.pop();
18         }
19         return value;
20     }
21
22     public int min() {
23         if (s2.isEmpty()) {
24             return Integer.MAX_VALUE;
25         } else {
26             return s2.peek();
27         }
28     }
29 }

```

Why might this be more space efficient? Suppose we had a very large stack and the first element inserted happened to be the minimum. In the first solution, we would be keeping n integers, where n is the size of the stack. In the second solution though, we store just a few pieces of data: a second stack with one element and the members within this stack.

3.3 Stack of Plates: Imagine a (literal) stack of plates. If the stack gets too high, it might topple. Therefore, in real life, we would likely start a new stack when the previous stack exceeds some threshold. Implement a data structure `SetOfStacks` that mimics this. `SetOfStacks` should be composed of several stacks and should create a new stack once the previous one exceeds capacity. `SetOfStacks.push()` and `SetOfStacks.pop()` should behave identically to a single stack (that is, `pop()` should return the same values as it would if there were just a single stack).

FOLLOW UP

Implement a function `popAt(int index)` which performs a pop operation on a specific sub-stack.

pg 99

SOLUTION

In this problem, we've been told what our data structure should look like:

```

1  class SetOfStacks {
2      ArrayList<Stack> stacks = new ArrayList<Stack>();
3      public void push(int v) { ... }

```

```
4     public int pop() { ... }
5 }
```

We know that `push()` should behave identically to a single stack, which means that we need `push()` to call `push()` on the last stack in the array of stacks. We have to be a bit careful here though: if the last stack is at capacity, we need to create a new stack. Our code should look something like this:

```
1 void push(int v) {
2     Stack last = getLastStack();
3     if (last != null && !last.isFull()) { // add to last stack
4         last.push(v);
5     } else { // must create new stack
6         Stack stack = new Stack(capacity);
7         stack.push(v);
8         stacks.add(stack);
9     }
10 }
```

What should `pop()` do? It should behave similarly to `push()` in that it should operate on the last stack. If the last stack is empty (after popping), then we should remove the stack from the list of stacks.

```
1 int pop() {
2     Stack last = getLastStack();
3     if (last == null) throw new EmptyStackException();
4     int v = last.pop();
5     if (last.size == 0) stacks.remove(stacks.size() - 1);
6     return v;
7 }
```

Follow Up: Implement `popAt(int index)`

This is a bit trickier to implement, but we can imagine a “rollover” system. If we pop an element from stack 1, we need to remove the *bottom* of stack 2 and push it onto stack 1. We then need to rollover from stack 3 to stack 2, stack 4 to stack 3, etc.

You could make an argument that, rather than “rolling over,” we should be okay with some stacks not being at full capacity. This would improve the time complexity (by a fair amount, with a large number of elements), but it might get us into tricky situations later on if someone assumes that all stacks (other than the last) operate at full capacity. There’s no “right answer” here; you should discuss this trade-off with your interviewer.

```
1 public class SetOfStacks {
2     ArrayList<Stack> stacks = new ArrayList<Stack>();
3     public int capacity;
4     public SetOfStacks(int capacity) {
5         this.capacity = capacity;
6     }
7
8     public Stack getLastStack() {
9         if (stacks.size() == 0) return null;
10        return stacks.get(stacks.size() - 1);
11    }
12
13    public void push(int v) { /* see earlier code */ }
14    public int pop() { /* see earlier code */ }
15    public boolean isEmpty() {
16        Stack last = getLastStack();
17        return last == null || last.isEmpty();
18    }
19 }
```

```

19
20 public int popAt(int index) {
21     return leftShift(index, true);
22 }
23
24 public int leftShift(int index, boolean removeTop) {
25     Stack stack = stacks.get(index);
26     int removed_item;
27     if (removeTop) removed_item = stack.pop();
28     else removed_item = stack.removeBottom();
29     if (stack.isEmpty()) {
30         stacks.remove(index);
31     } else if (stacks.size() > index + 1) {
32         int v = leftShift(index + 1, false);
33         stack.push(v);
34     }
35     return removed_item;
36 }
37 }
38
39 public class Stack {
40     private int capacity;
41     public Node top, bottom;
42     public int size = 0;
43
44     public Stack(int capacity) { this.capacity = capacity; }
45     public boolean isFull() { return capacity == size; }
46
47     public void join(Node above, Node below) {
48         if (below != null) below.above = above;
49         if (above != null) above.below = below;
50     }
51
52     public boolean push(int v) {
53         if (size >= capacity) return false;
54         size++;
55         Node n = new Node(v);
56         if (size == 1) bottom = n;
57         join(n, top);
58         top = n;
59         return true;
60     }
61
62     public int pop() {
63         Node t = top;
64         top = top.below;
65         size--;
66         return t.value;
67     }
68
69     public boolean isEmpty() {
70         return size == 0;
71     }
72
73     public int removeBottom() {
74         Node b = bottom;

```

```

75     bottom = bottom.above;
76     if (bottom != null) bottom.below = null;
77     size--;
78     return b.value;
79 }
80 }

```

This problem is not conceptually that tough, but it requires a lot of code to implement it fully. Your interviewer would not ask you to implement the entire code.

A good strategy on problems like this is to separate code into other methods, like a `leftShift` method that `popAt` can call. This will make your code cleaner and give you the opportunity to lay down the skeleton of the code before dealing with some of the details.

3.4 Queue via Stacks: Implement a `MyQueue` class which implements a queue using two stacks.

pg 99

SOLUTION

Since the major difference between a queue and a stack is the order (first-in first-out vs. last-in first-out), we know that we need to modify `peek()` and `pop()` to go in reverse order. We can use our second stack to reverse the order of the elements (by popping `s1` and pushing the elements on to `s2`). In such an implementation, on each `peek()` and `pop()` operation, we would pop everything from `s1` onto `s2`, perform the peek / pop operation, and then push everything back.

This will work, but if two pop / peeks are performed back-to-back, we're needlessly moving elements. We can implement a "lazy" approach where we let the elements sit in `s2` until we absolutely must reverse the elements.

In this approach, `stackNewest` has the newest elements on top and `stackOldest` has the oldest elements on top. When we dequeue an element, we want to remove the oldest element first, and so we dequeue from `stackOldest`. If `stackOldest` is empty, then we want to transfer all elements from `stackNewest` into this stack in reverse order. To insert an element, we push onto `stackNewest`, since it has the newest elements on top.

The code below implements this algorithm.

```

1  public class MyQueue<T> {
2      Stack<T> stackNewest, stackOldest;
3
4      public MyQueue() {
5          stackNewest = new Stack<T>();
6          stackOldest = new Stack<T>();
7      }
8
9      public int size() {
10         return stackNewest.size() + stackOldest.size();
11     }
12
13     public void add(T value) {
14         /* Push onto stackNewest, which always has the newest elements on top */
15         stackNewest.push(value);
16     }
17
18     /* Move elements from stackNewest into stackOldest. This is usually done so that
19     * we can do operations on stackOldest. */

```

```

20 private void shiftStacks() {
21     if (stackOldest.isEmpty()) {
22         while (!stackNewest.isEmpty()) {
23             stackOldest.push(stackNewest.pop());
24         }
25     }
26 }
27
28 public T peek() {
29     shiftStacks(); // Ensure stackOldest has the current elements
30     return stackOldest.peek(); // retrieve the oldest item.
31 }
32
33 public T remove() {
34     shiftStacks(); // Ensure stackOldest has the current elements
35     return stackOldest.pop(); // pop the oldest item.
36 }
37 }

```

During your actual interview, you may find that you forget the exact API calls. Don't stress too much if that happens to you. Most interviewers are okay with your asking for them to refresh your memory on little details. They're much more concerned with your big picture understanding.

3.5 Sort Stack: Write a program to sort a stack such that the smallest items are on the top. You can use an additional temporary stack, but you may not copy the elements into any other data structure (such as an array). The stack supports the following operations: push, pop, peek, and isEmpty.

pg 99

SOLUTION

One approach is to implement a rudimentary sorting algorithm. We search through the entire stack to find the minimum element and then push that onto a new stack. Then, we find the new minimum element and push that. This will actually require a total of three stacks: s1 is the original stack, s2 is the final sorted stack, and s3 acts as a buffer during our searching of s1. To search s1 for each minimum, we need to pop elements from s1 and push them onto the buffer, s3.

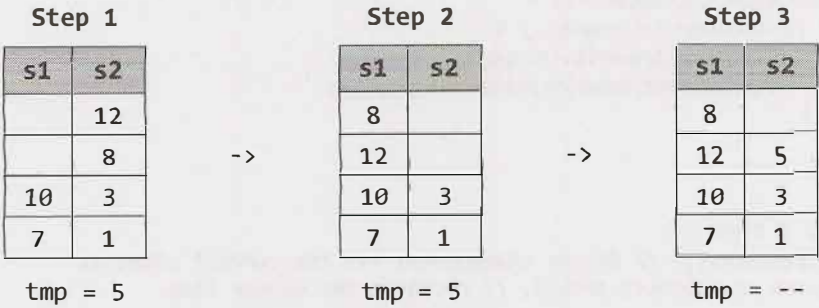
Unfortunately, this requires two additional stacks, and we can only use one. Can we do better? Yes.

Rather than searching for the minimum repeatedly, we can sort s1 by inserting each element from s1 in order into s2. How would this work?

Imagine we have the following stacks, where s2 is "sorted" and s1 is not:

s1	s2
	12
5	8
10	3
7	1

When we pop 5 from s1, we need to find the right place in s2 to insert this number. In this case, the correct place is on s2 just above 3. How do we get it there? We can do this by popping 5 from s1 and holding it in a temporary variable. Then, we move 12 and 8 over to s1 (by popping them from s2 and pushing them onto s1) and then push 5 onto s2.



Note that 8 and 12 are still in s1—and that’s okay! We just repeat the same steps for those two numbers as we did for 5, each time popping off the top of s1 and putting it into the “right place” on s2. (Of course, since 8 and 12 were moved from s2 to s1 precisely *because* they were larger than 5, the “right place” for these elements will be right on top of 5. We won’t need to muck around with s2’s other elements, and the inside of the below while loop will not be run when tmp is 8 or 12.)

```
1 void sort(Stack<Integer> s) {
2     Stack<Integer> r = new Stack<Integer>();
3     while(!s.isEmpty()) {
4         /* Insert each element in s in sorted order into r. */
5         int tmp = s.pop();
6         while(!r.isEmpty() && r.peek() > tmp) {
7             s.push(r.pop());
8         }
9         r.push(tmp);
10    }
11
12    /* Copy the elements from r back into s. */
13    while (!r.isEmpty()) {
14        s.push(r.pop());
15    }
16 }
```

This algorithm is $O(N^2)$ time and $O(N)$ space.

If we were allowed to use unlimited stacks, we could implement a modified quicksort or mergesort.

With the mergesort solution, we would create two extra stacks and divide the stack into two parts. We would recursively sort each stack, and then merge them back together in sorted order into the original stack. Note that this would require the creation of two additional stacks per level of recursion.

With the quicksort solution, we would create two additional stacks and divide the stack into the two stacks based on a pivot element. The two stacks would be recursively sorted, and then merged back together into the original stack. Like the earlier solution, this one involves creating two additional stacks per level of recursion.

3.6 Animal Shelter: An animal shelter, which holds only dogs and cats, operates on a strictly “first in, first out” basis. People must adopt either the “oldest” (based on arrival time) of all animals at the shelter, or they can select whether they would prefer a dog or a cat (and will receive the oldest animal of that type). They cannot select which specific animal they would like. Create the data structures to maintain this system and implement operations such as enqueue, dequeueAny, dequeueDog, and dequeueCat. You may use the built-in `LinkedList` data structure.

pg 99

SOLUTION

We could explore a variety of solutions to this problem. For instance, we could maintain a single queue. This would make `dequeueAny` easy, but `dequeueDog` and `dequeueCat` would require iteration through the queue to find the first dog or cat. This would increase the complexity of the solution and decrease the efficiency.

An alternative approach that is simple, clean and efficient is to simply use separate queues for dogs and cats, and to place them within a wrapper class called `AnimalQueue`. We then store some sort of timestamp to mark when each animal was enqueued. When we call `dequeueAny`, we peek at the heads of both the dog and cat queue and return the oldest.

```

1  abstract class Animal {
2      private int order;
3      protected String name;
4      public Animal(String n) { name = n; }
5      public void setOrder(int ord) { order = ord; }
6      public int getOrder() { return order; }
7
8      /* Compare orders of animals to return the older item. */
9      public boolean isOlderThan(Animal a) {
10         return this.order < a.getOrder();
11     }
12 }
13
14 class AnimalQueue {
15     LinkedList<Dog> dogs = new LinkedList<Dog>();
16     LinkedList<Cat> cats = new LinkedList<Cat>();
17     private int order = 0; // acts as timestamp
18
19     public void enqueue(Animal a) {
20         /* Order is used as a sort of timestamp, so that we can compare the insertion
21          * order of a dog to a cat. */
22         a.setOrder(order);
23         order++;
24
25         if (a instanceof Dog) dogs.addLast((Dog) a);
26         else if (a instanceof Cat) cats.addLast((Cat) a);
27     }
28
29     public Animal dequeueAny() {
30         /* Look at tops of dog and cat queues, and pop the queue with the oldest
31          * value. */
32         if (dogs.size() == 0) {
33             return dequeueCats();
34         } else if (cats.size() == 0) {
35             return dequeueDogs();
36         }

```

```
37
38     Dog dog = dogs.peek();
39     Cat cat = cats.peek();
40     if (dog.isOlderThan(cat)) {
41         return dequeueDogs();
42     } else {
43         return dequeueCats();
44     }
45 }
46
47 public Dog dequeueDogs() {
48     return dogs.poll();
49 }
50
51 public Cat dequeueCats() {
52     return cats.poll();
53 }
54 }
55
56 public class Dog extends Animal {
57     public Dog(String n) { super(n); }
58 }
59
60 public class Cat extends Animal {
61     public Cat(String n) { super(n); }
62 }
```

It is important that `Dog` and `Cat` both inherit from an `Animal` class since `dequeueAny()` needs to be able to support returning both `Dog` and `Cat` objects.

If we wanted, `order` could be a true timestamp with the actual date and time. The advantage of this is that we wouldn't have to set and maintain the numerical order. If we somehow wound up with two animals with the same timestamp, then (by definition) we don't have an older animal and we could return either one.

4

Solutions to Trees and Graphs

4.1 Route Between Nodes: Given a directed graph, design an algorithm to find out whether there is a route between two nodes.

pg 109

SOLUTION

This problem can be solved by just simple graph traversal, such as depth-first search or breadth-first search. We start with one of the two nodes and, during traversal, check if the other node is found. We should mark any node found in the course of the algorithm as “already visited” to avoid cycles and repetition of the nodes.

The code below provides an iterative implementation of breadth-first search.

```
1  enum State { Unvisited, Visited, Visiting; }
2
3  boolean search(Graph g, Node start, Node end) {
4      if (start == end) return true;
5
6      // operates as Queue
7      LinkedList<Node> q = new LinkedList<Node>();
8
9      for (Node u : g.getNodes()) {
10         u.state = State.Unvisited;
11     }
12     start.state = State.Visiting;
13     q.add(start);
14     Node u;
15     while (!q.isEmpty()) {
16         u = q.removeFirst(); // i.e., dequeue()
17         if (u != null) {
18             for (Node v : u.getAdjacent()) {
19                 if (v.state == State.Unvisited) {
20                     if (v == end) {
21                         return true;
22                     } else {
23                         v.state = State.Visiting;
24                         q.add(v);
25                     }
26                 }
27             }
28             u.state = State.Visited;
29         }
30     }
```

```
30     }
31     return false;
32 }
```

It may be worth discussing with your interviewer the tradeoffs between breadth-first search and depth-first search for this and other problems. For example, depth-first search is a bit simpler to implement since it can be done with simple recursion. Breadth-first search can also be useful to find the shortest path, whereas depth-first search may traverse one adjacent node very deeply before ever going onto the immediate neighbors.

4.2 Minimal Tree: Given a sorted (increasing order) array with unique integer elements, write an algorithm to create a binary search tree with minimal height.

pg 109

SOLUTION

To create a tree of minimal height, we need to match the number of nodes in the left subtree to the number of nodes in the right subtree as much as possible. This means that we want the root to be the middle of the array, since this would mean that half the elements would be less than the root and half would be greater than it.

We proceed with constructing our tree in a similar fashion. The middle of each subsection of the array becomes the root of the node. The left half of the array will become our left subtree, and the right half of the array will become the right subtree.

One way to implement this is to use a simple `root.insertNode(int v)` method which inserts the value `v` through a recursive process that starts with the root node. This will indeed construct a tree with minimal height but it will not do so very efficiently. Each insertion will require traversing the tree, giving a total cost of $O(N \log N)$ to the tree.

Alternatively, we can cut out the extra traversals by recursively using the `createMinimalBST` method. This method is passed just a subsection of the array and returns the root of a minimal tree for that array.

The algorithm is as follows:

1. Insert into the tree the middle element of the array.
2. Insert (into the left subtree) the left subarray elements.
3. Insert (into the right subtree) the right subarray elements.
4. Recurse.

The code below implements this algorithm.

```
1  TreeNode createMinimalBST(int array[]) {
2      return createMinimalBST(array, 0, array.length - 1);
3  }
4
5  TreeNode createMinimalBST(int arr[], int start, int end) {
6      if (end < start) {
7          return null;
8      }
9      int mid = (start + end) / 2;
10     TreeNode n = new TreeNode(arr[mid]);
11     n.left = createMinimalBST(arr, start, mid - 1);
12     n.right = createMinimalBST(arr, mid + 1, end);
13     return n;
14 }
```

```
14 }
```

Although this code does not seem especially complex, it can be very easy to make little off-by-one errors. Be sure to test these parts of the code very thoroughly.

4.3 List of Depths: Given a binary tree, design an algorithm which creates a linked list of all the nodes at each depth (e.g., if you have a tree with depth D , you'll have D linked lists).

pg 109

SOLUTION

Though we might think at first glance that this problem requires a level-by-level traversal, this isn't actually necessary. We can traverse the graph any way that we'd like, provided we know which level we're on as we do so.

We can implement a simple modification of the pre-order traversal algorithm, where we pass in `level + 1` to the next recursive call. The code below provides an implementation using depth-first search.

```
1 void createLevelLinkedList(TreeNode root, ArrayList<LinkedList<TreeNode>> lists,
2                               int level) {
3     if (root == null) return; // base case
4
5     LinkedList<TreeNode> list = null;
6     if (lists.size() == level) { // Level not contained in list
7         list = new LinkedList<TreeNode>();
8         /* Levels are always traversed in order. So, if this is the first time we've
9          * visited level i, we must have seen levels 0 through i - 1. We can
10        * therefore safely add the level at the end. */
11        lists.add(list);
12    } else {
13        list = lists.get(level);
14    }
15    list.add(root);
16    createLevelLinkedList(root.left, lists, level + 1);
17    createLevelLinkedList(root.right, lists, level + 1);
18 }
19
20 ArrayList<LinkedList<TreeNode>> createLevelLinkedList(TreeNode root) {
21     ArrayList<LinkedList<TreeNode>> lists = new ArrayList<LinkedList<TreeNode>>();
22     createLevelLinkedList(root, lists, 0);
23     return lists;
24 }
```

Alternatively, we can also implement a modification of breadth-first search. With this implementation, we want to iterate through the root first, then level 2, then level 3, and so on.

With each level i , we will have already fully visited all nodes on level $i - 1$. This means that to get which nodes are on level i , we can simply look at all children of the nodes of level $i - 1$.

The code below implements this algorithm.

```
1 ArrayList<LinkedList<TreeNode>> createLevelLinkedList(TreeNode root) {
2     ArrayList<LinkedList<TreeNode>> result = new ArrayList<LinkedList<TreeNode>>();
3     /* "Visit" the root */
4     LinkedList<TreeNode> current = new LinkedList<TreeNode>();
5     if (root != null) {
6         current.add(root);
7     }
```

```
8
9  while (current.size() > 0) {
10     result.add(current); // Add previous level
11     LinkedList<TreeNode> parents = current; // Go to next level
12     current = new LinkedList<TreeNode>();
13     for (TreeNode parent : parents) {
14         /* Visit the children */
15         if (parent.left != null) {
16             current.add(parent.left);
17         }
18         if (parent.right != null) {
19             current.add(parent.right);
20         }
21     }
22 }
23 return result;
24 }
```

One might ask which of these solutions is more efficient. Both run in $O(N)$ time, but what about the space efficiency? At first, we might want to claim that the second solution is more space efficient.

In a sense, that's correct. The first solution uses $O(\log N)$ recursive calls (in a balanced tree), each of which adds a new level to the stack. The second solution, which is iterative, does not require this extra space.

However, both solutions require returning $O(N)$ data. The extra $O(\log N)$ space usage from the recursive implementation is dwarfed by the $O(N)$ data that must be returned. So while the first solution may actually use more data, they are equally efficient when it comes to "big O."

4.4 Check Balanced: Implement a function to check if a binary tree is balanced. For the purposes of this question, a balanced tree is defined to be a tree such that the heights of the two subtrees of any node never differ by more than one.

pg 110

SOLUTION

In this question, we've been fortunate enough to be told exactly what balanced means: that for each node, the two subtrees differ in height by no more than one. We can implement a solution based on this definition. We can simply recurse through the entire tree, and for each node, compute the heights of each subtree.

```
1  int getHeight(TreeNode root) {
2      if (root == null) return -1; // Base case
3      return Math.max(getHeight(root.left), getHeight(root.right)) + 1;
4  }
5
6  boolean isBalanced(TreeNode root) {
7      if (root == null) return true; // Base case
8
9      int heightDiff = getHeight(root.left) - getHeight(root.right);
10     if (Math.abs(heightDiff) > 1) {
11         return false;
12     } else { // Recurse
13         return isBalanced(root.left) && isBalanced(root.right);
14     }
15 }
```

Although this works, it's not very efficient. On each node, we recurse through its entire subtree. This means that `getHeight` is called repeatedly on the same nodes. The algorithm is $O(N \log N)$ since each node is "touched" once per node above it.

We need to cut out some of the calls to `getHeight`.

If we inspect this method, we may notice that `getHeight` could actually check if the tree is balanced at the same time as it's checking heights. What do we do when we discover that the subtree isn't balanced? Just return an error code.

This improved algorithm works by checking the height of each subtree as we recurse down from the root. On each node, we recursively get the heights of the left and right subtrees through the `checkHeight` method. If the subtree is balanced, then `checkHeight` will return the actual height of the subtree. If the subtree is not balanced, then `checkHeight` will return an error code. We will immediately break and return an error code from the current call.

What do we use for an error code? The height of a null tree is generally defined to be -1, so that's not a great idea for an error code. Instead, we'll use `Integer.MIN_VALUE`.

The code below implements this algorithm.

```

1  int checkHeight(TreeNode root) {
2      if (root == null) return -1;
3
4      int leftHeight = checkHeight(root.left);
5      if (leftHeight == Integer.MIN_VALUE) return Integer.MIN_VALUE; // Pass error up
6
7      int rightHeight = checkHeight(root.right);
8      if (rightHeight == Integer.MIN_VALUE) return Integer.MIN_VALUE; // Pass error up
9
10     int heightDiff = leftHeight - rightHeight;
11     if (Math.abs(heightDiff) > 1) {
12         return Integer.MIN_VALUE; // Found error -> pass it back
13     } else {
14         return Math.max(leftHeight, rightHeight) + 1;
15     }
16 }
17
18 boolean isBalanced(TreeNode root) {
19     return checkHeight(root) != Integer.MIN_VALUE;
20 }
```

This code runs in $O(N)$ time and $O(H)$ space, where H is the height of the tree.

4.5 Validate BST: Implement a function to check if a binary tree is a binary search tree.

pg 110

SOLUTION

We can implement this solution in two different ways. The first leverages the in-order traversal, and the second builds off the property that `left <= current < right`.

Solution #1: In-Order Traversal

Our first thought might be to do an in-order traversal, copy the elements to an array, and then check to see if the array is sorted. This solution takes up a bit of extra memory, but it works—mostly.

The only problem is that it can't handle duplicate values in the tree properly. For example, the algorithm cannot distinguish between the two trees below (one of which is invalid) since they have the same in-order traversal.



However, if we assume that the tree cannot have duplicate values, then this approach works. The pseudo-code for this method looks something like:

```

1  int index = 0;
2  void copyBST(TreeNode root, int[] array) {
3      if (root == null) return;
4      copyBST(root.left, array);
5      array[index] = root.data;
6      index++;
7      copyBST(root.right, array);
8  }
9
10 boolean checkBST(TreeNode root) {
11     int[] array = new int[root.size];
12     copyBST(root, array);
13     for (int i = 1; i < array.length; i++) {
14         if (array[i] <= array[i - 1]) return false;
15     }
16     return true;
17 }
  
```

Note that it is necessary to keep track of the logical “end” of the array, since it would be allocated to hold all the elements.

When we examine this solution, we find that the array is not actually necessary. We never use it other than to compare an element to the previous element. So why not just track the last element we saw and compare it as we go?

The code below implements this algorithm.

```

1  Integer last_printed = null;
2  boolean checkBST(TreeNode n) {
3      if (n == null) return true;
4
5      // Check / recurse left
6      if (!checkBST(n.left)) return false;
7
8      // Check current
9      if (last_printed != null && n.data <= last_printed) {
10         return false;
11     }
12     last_printed = n.data;
13
14     // Check / recurse right
  
```

```

15  if (!checkBST(n.right)) return false;
16
17  return true; // All good!
18  }

```

We've used an Integer instead of `int` so that we can know when `last_printed` has been set to a value.

If you don't like the use of static variables, then you can tweak this code to use a wrapper class for the integer, as shown below.

```

1  class WrapInt {
2      public int value;
3  }

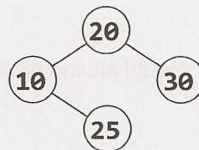
```

Or, if you're implementing this in C++ or another language that supports passing integers by reference, then you can simply do that.

Solution #2: The Min / Max Solution

In the second solution, we leverage the definition of the binary search tree.

What does it mean for a tree to be a binary search tree? We know that it must, of course, satisfy the condition `left.data <= current.data < right.data` for each node, but this isn't quite sufficient. Consider the following small tree:

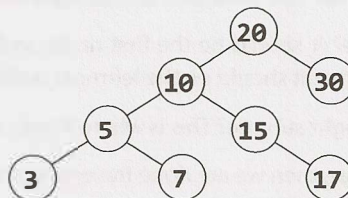


Although each node is bigger than its left node and smaller than its right node, this is clearly not a binary search tree since 25 is in the wrong place.

More precisely, the condition is that *all* left nodes must be less than or equal to the current node, which must be less than all the right nodes.

Using this thought, we can approach the problem by passing down the min and max values. As we iterate through the tree, we verify against progressively narrower ranges.

Consider the following sample tree:



We start with a range of (`min = NULL`, `max = NULL`), which the root obviously meets. (`NULL` indicates that there is no min or max.) We then branch left, checking that these nodes are within the range (`min = NULL`, `max = 20`). Then, we branch right, checking that the nodes are within the range (`min = 20`, `max = NULL`).

We proceed through the tree with this approach. When we branch left, the `max` gets updated. When we branch right, the `min` gets updated. If anything fails these checks, we stop and return false.

The time complexity for this solution is $O(N)$, where N is the number of nodes in the tree. We can prove that this is the best we can do, since any algorithm must touch all N nodes.

Due to the use of recursion, the space complexity is $O(\log N)$ on a balanced tree. There are up to $O(\log N)$ recursive calls on the stack since we may recurse up to the depth of the tree.

The recursive code for this is as follows:

```
1  boolean checkBST(TreeNode n) {
2      return checkBST(n, null, null);
3  }
4
5  boolean checkBST(TreeNode n, Integer min, Integer max) {
6      if (n == null) {
7          return true;
8      }
9      if ((min != null && n.data <= min) || (max != null && n.data > max)) {
10         return false;
11     }
12
13     if (!checkBST(n.left, min, n.data) || !checkBST(n.right, n.data, max)) {
14         return false;
15     }
16     return true;
17 }
```

Remember that in recursive algorithms, you should always make sure that your base cases, as well as your null cases, are well handled.

4.6 Successor: Write an algorithm to find the "next" node (i.e., in-order successor) of a given node in a binary search tree. You may assume that each node has a link to its parent.

pg 110

SOLUTION

Recall that an in-order traversal traverses the left subtree, then the current node, then the right subtree. To approach this problem, we need to think very, very carefully about what happens.

Let's suppose we have a hypothetical node. We know that the order goes left subtree, then current side, then right subtree. So, the next node we visit should be on the right side.

But which node on the right subtree? It should be the first node we'd visit if we were doing an in-order traversal of that subtree. This means that it should be the leftmost node on the right subtree. Easy enough!

But what if the node doesn't have a right subtree? This is where it gets a bit trickier.

If a node n doesn't have a right subtree, then we are done traversing n 's subtree. We need to pick up where we left off with n 's parent, which we'll call q .

If n was to the left of q , then the next node we should traverse should be q (again, since `left -> current -> right`).

If n were to the right of q , then we have fully traversed q 's subtree as well. We need to traverse upwards from q until we find a node x that we have *not* fully traversed. How do we know that we have not fully traversed a node x ? We know we have hit this case when we move from a left node to its parent. The left node is fully traversed, but its parent is not.

The pseudocode looks like this:

```

1 Node inorderSucc(Node n) {
2   if (n has a right subtree) {
3     return leftmost child of right subtree
4   } else {
5     while (n is a right child of n.parent) {
6       n = n.parent; // Go up
7     }
8     return n.parent; // Parent has not been traversed
9   }
10 }
```

But wait—what if we traverse all the way up the tree before finding a left child? This will happen only when we hit the very end of the in-order traversal. That is, if we're *already* on the far right of the tree, then there is no in-order successor. We should return null.

The code below implements this algorithm (and properly handles the null case).

```

1 TreeNode inorderSucc(TreeNode n) {
2   if (n == null) return null;
3
4   /* Found right children -> return leftmost node of right subtree. */
5   if (n.right != null) {
6     return leftMostChild(n.right);
7   } else {
8     TreeNode q = n;
9     TreeNode x = q.parent;
10    // Go up until we're on left instead of right
11    while (x != null && x.left != q) {
12      q = x;
13      x = x.parent;
14    }
15    return x;
16  }
17 }
18
19 TreeNode leftMostChild(TreeNode n) {
20   if (n == null) {
21     return null;
22   }
23   while (n.left != null) {
24     n = n.left;
25   }
26   return n;
27 }
```

This is not the most algorithmically complex problem in the world, but it can be tricky to code perfectly. In a problem like this, it's useful to sketch out pseudocode to carefully outline the different cases.

- 4.7 Build Order:** You are given a list of projects and a list of dependencies (which is a list of pairs of projects, where the second project is dependent on the first project). All of a project's dependencies must be built before the project is. Find a build order that will allow the projects to be built. If there is no valid build order, return an error.

EXAMPLE

Input:

projects: a, b, c, d, e, f

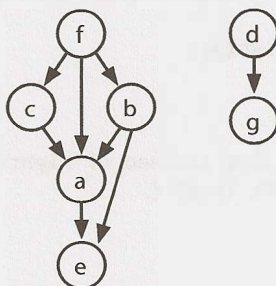
dependencies: (a, d), (f, b), (b, d), (f, a), (d, c)

Output: f, e, a, b, d, c

pg 110

SOLUTION

Visualizing the information as a graph probably works best. Be careful with the direction of the arrows. In the graph below, an arrow from d to g means that d must be compiled before g. You can also draw them in the opposite direction, but you need to be consistent and clear about what you mean. Let's draw a fresh example.



In drawing this example (which is *not* the example from the problem description), I looked for a few things.

- I wanted the nodes labeled somewhat randomly. If I had instead put a at the top, with b and c as children, then d and e, it could be misleading. The alphabetical order would match the compile order.
- I wanted a graph with multiple parts/components, since a connected graph is a bit of a special case.
- I wanted a graph where a node links to a node that cannot immediately follow it. For example, f links to a but a cannot immediately follow it (since b and c must come before a and after f).
- I wanted a larger graph since I need to figure out the pattern.
- I wanted nodes with multiple dependencies.

Now that we have a good example, let's get started with an algorithm.

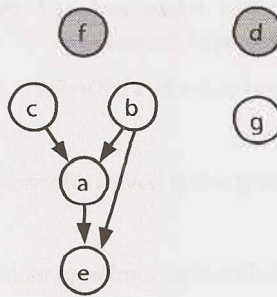
Solution #1

Where do we start? Are there any nodes that we can definitely compile immediately?

Yes. Nodes with no incoming edges can be built immediately since they don't depend on anything. Let's add all such nodes to the build order. In the earlier example, this means we have an order of f, d (or d, f).

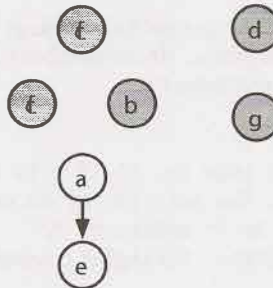
Once we've done that, it's irrelevant that some nodes are dependent on d and f since d and f have already been built. We can reflect this new state by removing d and f's outgoing edges.

build order: f, d



Next, we know that c, b, and g are free to build since they have no incoming edges. Let's build those and then remove their outgoing edges.

build order: f, d, c, b, g



Project a can be built next, so let's do that and remove its outgoing edges. This leaves just e. We build that next, giving us a complete build order.

build order: f, d, c, b, g, a, e

Did this algorithm work, or did we just get lucky? Let's think about the logic.

1. We first added the nodes with no incoming edges. If the set of projects can be built, there must be some "first" project, and that project can't have any dependencies. If a project has no dependencies (incoming edges), then we certainly can't break anything by building it first.
2. We removed all outgoing edges from these roots. This is reasonable. Once those root projects were built, it doesn't matter if another project depends on them.
3. After that, we found the nodes that *now* have no incoming edges. Using the same logic from steps 1 and 2, it's okay if we build these. Now we just repeat the same steps: find the nodes with no dependencies, add them to the build order, remove their outgoing edges, and repeat.
4. What if there are nodes remaining, but all have dependencies (incoming edges)? This means there's no way to build the system. We should return an error.

The implementation follows this approach very closely.

Initialization and setup:

1. Build a graph where each project is a node and its outgoing edges represent the projects that depend on it. That is, if A has an edge to B (A → B), it means B has a dependency on A and therefore A must be built before B. Each node also tracks the number of *incoming* edges.
2. Initialize a `buildOrder` array. Once we determine a project's build order, we add it to the array. We also continue to iterate through the array, using a `toBeProcessed` pointer to point to the next node to be fully processed.

- Find all the nodes with zero incoming edges and add those to a `buildOrder` array. Set a `toBeProcessed` pointer to the beginning of the array.

Repeat until `toBeProcessed` is at the end of the `buildOrder`:

- Read node at `toBeProcessed`.
 - » If node is null, then all remaining nodes have a dependency and we have detected a cycle.
- For each child of node:
 - » Decrement `child.dependencies` (the number of incoming edges).
 - » If `child.dependencies` is zero, add `child` to end of `buildOrder`.
- Increment `toBeProcessed`.

The code below implements this algorithm.

```
1  /* Find a correct build order. */
2  Project[] findBuildOrder(String[] projects, String[][] dependencies) {
3      Graph graph = buildGraph(projects, dependencies);
4      return orderProjects(graph.getNodes());
5  }
6
7  /* Build the graph, adding the edge (a, b) if b is dependent on a. Assumes a pair
8   * is listed in "build order". The pair (a, b) in dependencies indicates that b
9   * depends on a and a must be built before b. */
10 Graph buildGraph(String[] projects, String[][] dependencies) {
11     Graph graph = new Graph();
12     for (String project : projects) {
13         graph.createNode(project);
14     }
15
16     for (String[] dependency : dependencies) {
17         String first = dependency[0];
18         String second = dependency[1];
19         graph.addEdge(first, second);
20     }
21
22     return graph;
23 }
24
25 /* Return a list of the projects a correct build order.*/
26 Project[] orderProjects(ArrayList<Project> projects) {
27     Project[] order = new Project[projects.size()];
28
29     /* Add "roots" to the build order first.*/
30     int endOfList = addNonDependent(order, projects, 0);
31
32     int toBeProcessed = 0;
33     while (toBeProcessed < order.length) {
34         Project current = order[toBeProcessed];
35
36         /* We have a circular dependency since there are no remaining projects with
37          * zero dependencies. */
38         if (current == null) {
39             return null;
40         }
41     }
```

```

42     /* Remove myself as a dependency. */
43     ArrayList<Project> children = current.getChildren();
44     for (Project child : children) {
45         child.decrementDependencies();
46     }
47
48     /* Add children that have no one depending on them. */
49     endOfList = addNonDependent(order, children, endOfList);
50     toBeProcessed++;
51 }
52
53 return order;
54 }
55
56 /* A helper function to insert projects with zero dependencies into the order
57 * array, starting at index offset. */
58 int addNonDependent(Project[] order, ArrayList<Project> projects, int offset) {
59     for (Project project : projects) {
60         if (project.getNumberDependencies() == 0) {
61             order[offset] = project;
62             offset++;
63         }
64     }
65     return offset;
66 }
67
68 public class Graph {
69     private ArrayList<Project> nodes = new ArrayList<Project>();
70     private HashMap<String, Project> map = new HashMap<String, Project>();
71
72     public Project getNode(String name) {
73         if (!map.containsKey(name)) {
74             Project node = new Project(name);
75             nodes.add(node);
76             map.put(name, node);
77         }
78
79         return map.get(name);
80     }
81
82     public void addEdge(String startName, String endName) {
83         Project start = getNode(startName);
84         Project end = getNode(endName);
85         start.addNeighbor(end);
86     }
87
88     public ArrayList<Project> getNodes() { return nodes; }
89 }
90
91 public class Project {
92     private ArrayList<Project> children = new ArrayList<Project>();
93     private HashMap<String, Project> map = new HashMap<String, Project>();
94     private String name;
95     private int dependencies = 0;
96
97     public Project(String n) { name = n; }

```

```

98
99 public void addNeighbor(Project node) {
100     if (!map.containsKey(node.getName())) {
101         children.add(node);
102         map.put(node.getName(), node);
103         node.incrementDependencies();
104     }
105 }
106
107 public void incrementDependencies() { dependencies++; }
108 public void decrementDependencies() { dependencies--; }
109
110 public String getName() { return name; }
111 public ArrayList<Project> getChildren() { return children; }
112 public int getNumberDependencies() { return dependencies; }
113 }

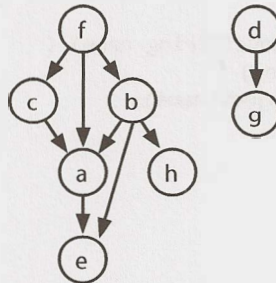
```

This solution takes $O(P + D)$ time, where P is the number of projects and D is the number of dependency pairs.

Note: You might recognize this as the topological sort algorithm on page 632. We've rederived this from scratch. Most people won't know this algorithm and it's reasonable for an interviewer to expect you to be able to derive it.

Solution #2

Alternatively, we can use depth-first search (DFS) to find the build path.



Suppose we picked an arbitrary node (say b) and performed a depth-first search on it. When we get to the end of a path and can't go any further (which will happen at h and e), we know that those terminating nodes can be the last projects to be built. No projects depend on them.

```

DFS(b) // Step 1
  DFS(h) // Step 2
    build order = ..., h // Step 3
  DFS(a) // Step 4
    DFS(e) // Step 5
      build order = ..., e, h // Step 6
    ... // Step 7+
  ...

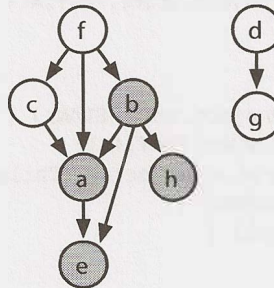
```

Now, consider what happens at node a when we return from the DFS of e . We know a 's children need to appear after a in the build order. So, once we return from searching a 's children (and therefore they have been added), we can choose to add a to the front of the build order.

Once we return from *a*, and complete the DFS of *b*'s other children, then everything that must appear after *b* is in the list. Add *b* to the front.

```
DFS(b)                                // Step 1
  DFS(h)                              // Step 2
    build order = ..., h              // Step 3
  DFS(a)                              // Step 4
    DFS(e)                            // Step 5
      build order = ..., e, h         // Step 6
    build order = ..., a, e, h        // Step 7
  DFS(e) -> return                    // Step 8
  build order = ..., b, a, e, h       // Step 9
```

Let's mark these nodes as having been built too, just in case someone else needs to build them.



Now what? We can start with any old node again, doing a DFS on it and then adding the node to the front of the build queue when the DFS is completed.

```
DFS(d)
  DFS(g)
    build order = ..., g, b, a, e, h
  build order = ..., d, g, b, a, e, h

DFS(f)
  DFS(c)
    build order = ..., c, d, g, b, a, e, h
  build order = f, c, d, g, b, a, e, h
```

In an algorithm like this, we should think about the issue of cycles. There is no possible build order if there is a cycle. But still, we don't want to get stuck in an infinite loop just because there's no possible solution.

A cycle will happen if, while doing a DFS on a node, we run back into the same path. What we need therefore is a signal that indicates "I'm still processing this node, so if you see the node again, we have a problem."

What we can do for this is to mark each node as a "partial" (or "is visiting") state just before we start the DFS on it. If we see any node whose state is `partial`, then we know we have a problem. When we're done with this node's DFS, we need to update the state.

We also need a state to indicate "I've already processed/built this node" so we don't re-build the node. Our state therefore can have three options: `COMPLETED`, `PARTIAL`, and `BLANK`.

The code below implements this algorithm.

```
1 Stack<Project> findBuildOrder(String[] projects, String[][] dependencies) {
2   Graph graph = buildGraph(projects, dependencies);
3   return orderProjects(graph.getNodes());
4 }
```

```

5
6 Stack<Project> orderProjects(ArrayList<Project> projects) {
7     Stack<Project> stack = new Stack<Project>();
8     for (Project project : projects) {
9         if (project.getState() == Project.State.BLANK) {
10             if (!doDFS(project, stack)) {
11                 return null;
12             }
13         }
14     }
15     return stack;
16 }
17
18 boolean doDFS(Project project, Stack<Project> stack) {
19     if (project.getState() == Project.State.PARTIAL) {
20         return false; // Cycle
21     }
22
23     if (project.getState() == Project.State.BLANK) {
24         project.setState(Project.State.PARTIAL);
25         ArrayList<Project> children = project.getChildren();
26         for (Project child : children) {
27             if (!doDFS(child, stack)) {
28                 return false;
29             }
30         }
31         project.setState(Project.State.COMPLETE);
32         stack.push(project);
33     }
34     return true;
35 }
36
37 /* Same as before */
38 Graph buildGraph(String[] projects, String[][] dependencies) {...}
39 public class Graph {}
40
41 /* Essentially equivalent to earlier solution, with state info added and
42  * dependency count removed. */
43 public class Project {
44     public enum State {COMPLETE, PARTIAL, BLANK};
45     private State state = State.BLANK;
46     public State getState() { return state; }
47     public void setState(State st) { state = st; }
48     /* Duplicate code removed for brevity */
49 }

```

Like the earlier algorithm, this solution is $O(P+D)$ time, where P is the number of projects and D is the number of dependency pairs.

By the way, this problem is called **topological sort**: linearly ordering the vertices in a graph such that for every edge (a, b) , a appears before b in the linear order.

4.8 First Common Ancestor: Design an algorithm and write code to find the first common ancestor of two nodes in a binary tree. Avoid storing additional nodes in a data structure. NOTE: This is not necessarily a binary search tree.

pg 110

SOLUTION

If this were a binary search tree, we could modify the `find` operation for the two nodes and see where the paths diverge. Unfortunately, this is not a binary search tree, so we must try other approaches.

Let's assume we're looking for the common ancestor of nodes `p` and `q`. One question to ask here is if each node in our tree has a link to its parents.

Solution #1: With Links to Parents

If each node has a link to its parent, we could trace `p` and `q`'s paths up until they intersect. This is essentially the same problem as question 2.7 which find the intersection of two linked lists. The "linked list" in this case is the path from each node up to the root. (Review this solution on page 221.)

```

1  TreeNode commonAncestor(TreeNode p, TreeNode q) {
2      int delta = depth(p) - depth(q); // get difference in depths
3      TreeNode first = delta > 0 ? q : p; // get shallower node
4      TreeNode second = delta > 0 ? p : q; // get deeper node
5      second = goUpBy(second, Math.abs(delta)); // move deeper node up
6
7      /* Find where paths intersect. */
8      while (first != second && first != null && second != null) {
9          first = first.parent;
10         second = second.parent;
11     }
12     return first == null || second == null ? null : first;
13 }
14
15 TreeNode goUpBy(TreeNode node, int delta) {
16     while (delta > 0 && node != null) {
17         node = node.parent;
18         delta--;
19     }
20     return node;
21 }
22
23 int depth(TreeNode node) {
24     int depth = 0;
25     while (node != null) {
26         node = node.parent;
27         depth++;
28     }
29     return depth;
30 }

```

This approach will take $O(d)$ time, where d is the depth of the deeper node.

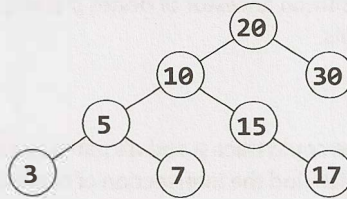
Solution #2: With Links to Parents (Better Worst-Case Runtime)

Similar to the earlier approach, we could trace `p`'s path upwards and check if any of the nodes cover `q`. The first node that covers `q` (we already know that every node on this path will cover `p`) must be the first common ancestor.

Observe that we don't need to re-check the entire subtree. As we move from a node x to its parent y , all the nodes under x have already been checked for q . Therefore, we only need to check the new nodes “uncovered”, which will be the nodes under x 's sibling.

For example, suppose we're looking for the first common ancestor of node $p = 7$ and node $q = 17$. When we go to $p.parent$ (5), we uncover the subtree rooted at 3. We therefore need to search this subtree for q .

Next, we go to node 10, uncovering the subtree rooted at 15. We check this subtree for node 17 and—voila—there it is.



To implement this, we can just traverse upwards from p , storing the parent and the sibling node in a variable. (The sibling node is always a child of parent and refers to the newly uncovered subtree.) At each iteration, sibling gets set to the old parent's sibling node and parent gets set to parent.parent.

```

1  TreeNode commonAncestor(TreeNode root, TreeNode p, TreeNode q) {
2      /* Check if either node is not in the tree, or if one covers the other. */
3      if (!covers(root, p) || !covers(root, q)) {
4          return null;
5      } else if (covers(p, q)) {
6          return p;
7      } else if (covers(q, p)) {
8          return q;
9      }
10
11     /* Traverse upwards until you find a node that covers q. */
12     TreeNode sibling = getSibling(p);
13     TreeNode parent = p.parent;
14     while (!covers(sibling, q)) {
15         sibling = getSibling(parent);
16         parent = parent.parent;
17     }
18     return parent;
19 }
20
21 boolean covers(TreeNode root, TreeNode p) {
22     if (root == null) return false;
23     if (root == p) return true;
24     return covers(root.left, p) || covers(root.right, p);
25 }
26
27 TreeNode getSibling(TreeNode node) {
28     if (node == null || node.parent == null) {
29         return null;
30     }
31
32     TreeNode parent = node.parent;
  
```

```

33     return parent.left == node ? parent.right : parent.left;
34 }

```

This algorithm takes $O(t)$ time, where t is the size of the subtree for the first common ancestor. In the worst case, this will be $O(n)$, where n is the number of nodes in the tree. We can derive this runtime by noticing that each node in that subtree is searched once.

Solution #3: Without Links to Parents

Alternatively, you could follow a chain in which p and q are on the same side. That is, if p and q are both on the left of the node, branch left to look for the common ancestor. If they are both on the right, branch right to look for the common ancestor. When p and q are no longer on the same side, you must have found the first common ancestor.

The code below implements this approach.

```

1  TreeNode commonAncestor(TreeNode root, TreeNode p, TreeNode q) {
2      /* Error check - one node is not in the tree. */
3      if (!covers(root, p) || !covers(root, q)) {
4          return null;
5      }
6      return ancestorHelper(root, p, q);
7  }
8
9  TreeNode ancestorHelper(TreeNode root, TreeNode p, TreeNode q) {
10     if (root == null || root == p || root == q) {
11         return root;
12     }
13
14     boolean pIsOnLeft = covers(root.left, p);
15     boolean qIsOnLeft = covers(root.left, q);
16     if (pIsOnLeft != qIsOnLeft) { // Nodes are on different side
17         return root;
18     }
19     TreeNode childSide = pIsOnLeft ? root.left : root.right;
20     return ancestorHelper(childSide, p, q);
21 }
22
23 boolean covers(TreeNode root, TreeNode p) {
24     if (root == null) return false;
25     if (root == p) return true;
26     return covers(root.left, p) || covers(root.right, p);
27 }

```

This algorithm runs in $O(n)$ time on a balanced tree. This is because `covers` is called on $2n$ nodes in the first call (n nodes for the left side, and n nodes for the right side). After that, the algorithm branches left or right, at which point `covers` will be called on $\frac{2n}{2}$ nodes, then $\frac{2n}{4}$, and so on. This results in a runtime of $O(n)$.

We know at this point that we cannot do better than that in terms of the asymptotic runtime since we need to potentially look at every node in the tree. However, we may be able to improve it by a constant multiple.

Solution #4: Optimized

Although Solution #3 is optimal in its runtime, we may recognize that there is still some inefficiency in how it operates. Specifically, `covers` searches all nodes under `root` for p and q , including the nodes in each subtree (`root.left` and `root.right`). Then, it picks one of those subtrees and searches all of its nodes. Each subtree is searched over and over again.

We may recognize that we should only need to search the entire tree once to find *p* and *q*. We should then be able to “bubble up” the findings to earlier nodes in the stack. The basic logic is the same as the earlier solution.

We recurse through the entire tree with a function called `commonAncestor(TreeNode root, TreeNode p, TreeNode q)`. This function returns values as follows:

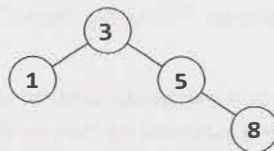
- Returns *p*, if root’s subtree includes *p* (and not *q*).
- Returns *q*, if root’s subtree includes *q* (and not *p*).
- Returns `null`, if neither *p* nor *q* are in root’s subtree.
- Else, returns the common ancestor of *p* and *q*.

Finding the common ancestor of *p* and *q* in the final case is easy. When `commonAncestor(n.left, p, q)` and `commonAncestor(n.right, p, q)` both return non-`null` values (indicating that *p* and *q* were found in different subtrees), then *n* will be the common ancestor.

The code below offers an initial solution, but it has a bug. Can you find it?

```
1  /* The below code has a bug. */
2  TreeNode commonAncestor(TreeNode root, TreeNode p, TreeNode q) {
3      if (root == null) return null;
4      if (root == p && root == q) return root;
5
6      TreeNode x = commonAncestor(root.left, p, q);
7      if (x != null && x != p && x != q) { // Already found ancestor
8          return x;
9      }
10
11     TreeNode y = commonAncestor(root.right, p, q);
12     if (y != null && y != p && y != q) { // Already found ancestor
13         return y;
14     }
15
16     if (x != null && y != null) { // p and q found in diff. subtrees
17         return root; // This is the common ancestor
18     } else if (root == p || root == q) {
19         return root;
20     } else {
21         return x == null ? y : x; /* return the non-null value */
22     }
23 }
```

The problem with this code occurs in the case where a node is not contained in the tree. For example, look at the following tree:



Suppose we call `commonAncestor(node 3, node 5, node 7)`. Of course, node 7 does not exist—and that’s where the issue will come in. The calling order looks like:

```
1  commonAnc(node 3, node 5, node 7)    // --> 5
2    calls commonAnc(node 1, node 5, node 7) // --> null
```

```

3    calls commonAnc(node 5, node 5, node 7)    // --> 5
4    calls commonAnc(node 8, node 5, node 7)    // --> null

```

In other words, when we call `commonAncestor` on the right subtree, the code will return node 5, just as it should. The problem is that, in finding the common ancestor of `p` and `q`, the calling function can't distinguish between the two cases:

- Case 1: `p` is a child of `q` (or, `q` is a child of `p`)
- Case 2: `p` is in the tree and `q` is not (or, `q` is in the tree and `p` is not)

In either of these cases, `commonAncestor` will return `p`. In the first case, this is the correct return value, but in the second case, the return value should be `null`.

We somehow need to distinguish between these two cases, and this is what the code below does. This code solves the problem by returning two values: the node itself and a flag indicating whether this node is actually the common ancestor.

```

1  class Result {
2      public TreeNode node;
3      public boolean isAncestor;
4      public Result(TreeNode n, boolean isAnc) {
5          node = n;
6          isAncestor = isAnc;
7      }
8  }
9
10  TreeNode commonAncestor(TreeNode root, TreeNode p, TreeNode q) {
11      Result r = commonAncestorHelper(root, p, q);
12      if (r.isAncestor) {
13          return r.node;
14      }
15      return null;
16  }
17
18  Result commonAncHelper(TreeNode root, TreeNode p, TreeNode q) {
19      if (root == null) return new Result(null, false);
20
21      if (root == p && root == q) {
22          return new Result(root, true);
23      }
24
25      Result rx = commonAncHelper(root.left, p, q);
26      if (rx.isAncestor) { // Found common ancestor
27          return rx;
28      }
29
30      Result ry = commonAncHelper(root.right, p, q);
31      if (ry.isAncestor) { // Found common ancestor
32          return ry;
33      }
34
35      if (rx.node != null && ry.node != null) {
36          return new Result(root, true); // This is the common ancestor
37      } else if (root == p || root == q) {
38          /* If we're currently at p or q, and we also found one of those nodes in a
39           * subtree, then this is truly an ancestor and the flag should be true. */
40          boolean isAncestor = rx.node != null || ry.node != null;

```

```

41     return new Result(root, isAncestor);
42   } else {
43     return new Result(rx.node!=null ? rx.node : ry.node, false);
44   }
45 }

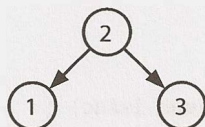
```

Of course, as this issue only comes up when p or q is not actually in the tree, an alternative solution would be to first search through the entire tree to make sure that both nodes exist.

4.9 BST Sequences: A binary search tree was created by traversing through an array from left to right and inserting each element. Given a binary search tree with distinct elements, print all possible arrays that could have led to this tree.

EXAMPLE

Input:

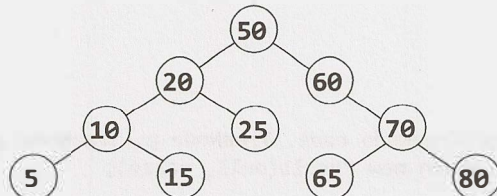


Output: {2, 1, 3}, {2, 3, 1}

pg 110

SOLUTION

It's useful to kick off this question with a good example.



We should also think about the ordering of items in a binary search tree. Given a node, all nodes on its left must be less than all nodes on its right. Once we reach a place without a node, we insert the new value there.

What this means is that the very first element in our array must have been a 50 in order to create the above tree. If it were anything else, then that value would have been the root instead.

What else can we say? Some people jump to the conclusion that everything on the left must have been inserted before elements on the right, but that's not actually true. In fact, the reverse is true: the order of the left or right items doesn't matter.

Once the 50 is inserted, all items less than 50 will be routed to the left and all items greater than 50 will be routed to the right. The 60 or the 20 could be inserted first, and it wouldn't matter.

Let's think about this problem recursively. If we had all arrays that could have created the subtree rooted at 20 (call this `arraySet20`), and all arrays that could have created the subtree rooted at 60 (call this `arraySet60`), how would that give us the full answer? We could just "weave" each array from `arraySet20` with each array from `arraySet60`—and then prepend each array with a 50.

Here's what we mean by weaving. We are merging two arrays in all possible ways, while keeping the elements within each array in the same relative order.

```
array1: {1, 2}
array2: {3, 4}
weaved: {1, 2, 3, 4}, {1, 3, 2, 4}, {1, 3, 4, 2},
        {3, 1, 2, 4}, {3, 1, 4, 2}, {3, 4, 1, 2}
```

Note that, as long as there aren't any duplicates in the original array sets, we won't have to worry that weaving will create duplicates.

The last piece to talk about here is how the weaving works. Let's think recursively about how to weave {1, 2, 3} and {4, 5, 6}. What are the subproblems?

- Prepend a 1 to all weaves of {2, 3} and {4, 5, 6}.
- Prepend a 4 to all weaves of {1, 2, 3} and {5, 6}.

To implement this, we'll store each as linked lists. This will make it easy to add and remove elements. When we recurse, we'll push the prefixed elements down the recursion. When `first` or `second` are empty, we add the remainder to `prefix` and store the result.

It works something like this:

```
weave(first, second, prefix):
    weave({1, 2}, {3, 4}, {})
        weave({2}, {3, 4}, {1})
            weave({}, {3, 4}, {1, 2})
                {1, 2, 3, 4}
            weave({2}, {4}, {1, 3})
                weave({}, {4}, {1, 3, 2})
                    {1, 3, 2, 4}
                weave({2}, {}, {1, 3, 4})
                    {1, 3, 4, 2}
        weave({1, 2}, {4}, {3})
            weave({2}, {4}, {3, 1})
                weave({}, {4}, {3, 1, 2})
                    {3, 1, 2, 4}
                weave({2}, {}, {3, 1, 4})
                    {3, 1, 4, 2}
            weave({1, 2}, {}, {3, 4})
                {3, 4, 1, 2}
```

Now, let's think through the implementation of removing, say, 1 from {1, 2} and recursing. We need to be careful about modifying this list, since a later recursive call (e.g., `weave({1, 2}, {4}, {3})`) might need the 1 still in {1, 2}.

We could clone the list when we recurse, so that we only modify the recursive calls. Or, we could modify the list, but then "revert" the changes after we're done with recursing.

We've chosen to implement it the latter way. Since we're keeping the same reference to `first`, `second`, and `prefix` the entire way down the recursive call stack, then we'll need to clone `prefix` just before we store the complete result.

```
1 ArrayList<LinkedList<Integer>> allSequences(TreeNode node) {
2     ArrayList<LinkedList<Integer>> result = new ArrayList<LinkedList<Integer>>();
3
4     if (node == null) {
5         result.add(new LinkedList<Integer>());
6         return result;
7     }
```

```

7     }
8
9     LinkedList<Integer> prefix = new LinkedList<Integer>();
10    prefix.add(node.data);
11
12    /* Recurse on left and right subtrees. */
13    ArrayList<LinkedList<Integer>> leftSeq = allSequences(node.left);
14    ArrayList<LinkedList<Integer>> rightSeq = allSequences(node.right);
15
16    /* Weave together each list from the left and right sides. */
17    for (LinkedList<Integer> left : leftSeq) {
18        for (LinkedList<Integer> right : rightSeq) {
19            ArrayList<LinkedList<Integer>> weaved =
20                new ArrayList<LinkedList<Integer>>();
21            weaveLists(left, right, weaved, prefix);
22            result.addAll(weaved);
23        }
24    }
25    return result;
26 }
27
28 /* Weave lists together in all possible ways. This algorithm works by removing the
29 * head from one list, recursing, and then doing the same thing with the other
30 * list. */
31 void weaveLists(LinkedList<Integer> first, LinkedList<Integer> second,
32                ArrayList<LinkedList<Integer>> results, LinkedList<Integer> prefix) {
33     /* One list is empty. Add remainder to [a cloned] prefix and store result. */
34     if (first.size() == 0 || second.size() == 0) {
35         LinkedList<Integer> result = (LinkedList<Integer>) prefix.clone();
36         result.addAll(first);
37         result.addAll(second);
38         results.add(result);
39         return;
40     }
41
42     /* Recurse with head of first added to the prefix. Removing the head will damage
43     * first, so we'll need to put it back where we found it afterwards. */
44     int headFirst = first.removeFirst();
45     prefix.addLast(headFirst);
46     weaveLists(first, second, results, prefix);
47     prefix.removeLast();
48     first.addFirst(headFirst);
49
50     /* Do the same thing with second, damaging and then restoring the list.*/
51     int headSecond = second.removeFirst();
52     prefix.addLast(headSecond);
53     weaveLists(first, second, results, prefix);
54     prefix.removeLast();
55     second.addFirst(headSecond);
56 }

```

Some people struggle with this problem because there are two different recursive algorithms that must be designed and implemented. They get confused with how the algorithms should interact with each other and they try to juggle both in their heads.

If this sounds like you, try this: *trust and focus*. *Trust* that one method does the right thing when implementing an independent method, and *focus* on the one thing that this independent method needs to do.

Look at `weaveLists`. It has a specific job: to weave two lists together and return a list of all possible weaves. The existence of `allSequences` is irrelevant. Focus on the task that `weaveLists` has to do and design this algorithm.

As you're implementing `allSequences` (whether you do this before or after `weaveLists`), trust that `weaveLists` will do the right thing. Don't concern yourself with the particulars of how `weaveLists` operates while implementing something that is essentially independent. Focus on what you're doing while you're doing it.

In fact, this is good advice in general when you're confused during whiteboard coding. Have a good understanding of what a particular function should do ("okay, this function is going to return a list of ____"). You should verify that it's really doing what you think. But when you're not dealing with that function, focus on the one you are dealing with and trust that the others do the right thing. It's often too much to keep the implementations of multiple algorithms straight in your head.

4.10 Check Subtree: $T1$ and $T2$ are two very large binary trees, with $T1$ much bigger than $T2$. Create an algorithm to determine if $T2$ is a subtree of $T1$.

A tree $T2$ is a subtree of $T1$ if there exists a node n in $T1$ such that the subtree of n is identical to $T2$. That is, if you cut off the tree at node n , the two trees would be identical.

pg 111

SOLUTION

In problems like this, it's useful to attempt to solve the problem assuming that there is just a small amount of data. This will give us a basic idea of an approach that might work.

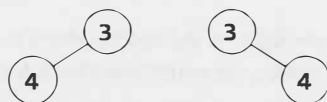
The Simple Approach

In this smaller, simpler problem, we could consider comparing string representations of traversals of each tree. If $T2$ is a subtree of $T1$, then $T2$'s traversal should be a substring of $T1$. Is the reverse true? If so, should we use an in-order traversal or a pre-order traversal?

An in-order traversal will definitely not work. After all, consider a scenario in which we were using binary search trees. A binary search tree's in-order traversal always prints out the values in sorted order. Therefore, two binary search trees with the same values will always have the same in-order traversals, even if their structure is different.

What about a pre-order traversal? This is a bit more promising. At least in this case we know certain things, like the first element in the pre-order traversal is the root node. The left and right elements will follow.

Unfortunately, trees with different structures could still have the same pre-order traversal.

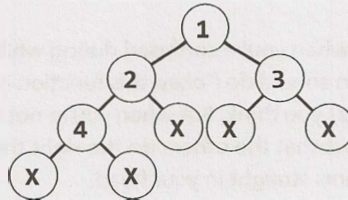


There's a simple fix though. We can store NULL nodes in the pre-order traversal string as a special character, like an 'X'. (We'll assume that the binary trees contain only integers.) The left tree would have the traversal $\{3, 4, X\}$ and the right tree will have the traversal $\{3, X, 4\}$.

Observe that, as long as we represent the NULL nodes, the pre-order traversal of a tree is unique. That is, if two trees have the same pre-order traversal, then we know they are identical trees in values and structure.

To see this, consider reconstructing a tree from its pre-order traversal (with NULL nodes indicated). For example: 1, 2, 4, X, X, X, 3, X, X.

The root is 1, and its left node, 2, follows it. 2.left must be 4. 4 must have two NULL nodes (since it is followed by two Xs). 4 is complete, so we move back up to its parent, 2. 2.right is another X (NULL). 1's left subtree is now complete, so we move to 1's right child. We place a 3 with two NULL children there. The tree is now complete.



This whole process was deterministic, as it will be on any other tree. A pre-order traversal always starts at the root and, from there, the path we take is entirely defined by the traversal. Therefore, two trees are identical if they have the same pre-order traversal.

Now consider the subtree problem. If T2's pre-order traversal is a substring of T1's pre-order traversal, then T2's root element must be found in T1. If we do a pre-order traversal from this element in T1, we will follow an identical path to T2's traversal. Therefore, T2 is a subtree of T1.

Implementing this is quite straightforward. We just need to construct and compare the pre-order traversals.

```
1  boolean containsTree(TreeNode t1, TreeNode t2) {
2      StringBuilder string1 = new StringBuilder();
3      StringBuilder string2 = new StringBuilder();
4
5      getOrderString(t1, string1);
6      getOrderString(t2, string2);
7
8      return string1.indexOf(string2.toString()) != -1;
9  }
10
11 void getOrderString(TreeNode node, StringBuilder sb) {
12     if (node == null) {
13         sb.append("X");           // Add null indicator
14         return;
15     }
16     sb.append(node.data + " ");   // Add root
17     getOrderString(node.left, sb); // Add left
18     getOrderString(node.right, sb); // Add right
19 }
```

This approach takes $O(n + m)$ time and $O(n + m)$ space, where n and m are the number of nodes in T1 and T2, respectively. Given millions of nodes, we might want to reduce the space complexity.

The Alternative Approach

An alternative approach is to search through the larger tree, T1. Each time a node in T1 matches the root of T2, call `matchTree`. The `matchTree` method will compare the two subtrees to see if they are identical.

Analyzing the runtime is somewhat complex. A naive answer would be to say that it is $O(nm)$ time, where n is the number of nodes in T1 and m is the number of nodes in T2. While this is technically correct, a little more thought can produce a tighter bound.

We do not actually call `matchTree` on every node in `T1`. Rather, we call it k times, where k is the number of occurrences of `T2`'s root in `T1`. The runtime is closer to $O(n + km)$.

In fact, even that overstates the runtime. Even if the root were identical, we exit `matchTree` when we find a difference between `T1` and `T2`. We therefore probably do not actually look at m nodes on each call of `matchTree`.

The code below implements this algorithm.

```

1  boolean containsTree(TreeNode t1, TreeNode t2) {
2      if (t2 == null) return true; // The empty tree is always a subtree
3      return subTree(t1, t2);
4  }
5
6  boolean subTree(TreeNode r1, TreeNode r2) {
7      if (r1 == null) {
8          return false; // big tree empty & subtree still not found.
9      } else if (r1.data == r2.data && matchTree(r1, r2)) {
10         return true;
11     }
12     return subTree(r1.left, r2) || subTree(r1.right, r2);
13 }
14
15 boolean matchTree(TreeNode r1, TreeNode r2) {
16     if (r1 == null && r2 == null) {
17         return true; // nothing left in the subtree
18     } else if (r1 == null || r2 == null) {
19         return false; // exactly tree is empty, therefore trees don't match
20     } else if (r1.data != r2.data) {
21         return false; // data doesn't match
22     } else {
23         return matchTree(r1.left, r2.left) && matchTree(r1.right, r2.right);
24     }
25 }

```

When might the simple solution be better, and when might the alternative approach be better? This is a great conversation to have with your interviewer. Here are a few thoughts on that matter:

1. The simple solution takes $O(n + m)$ memory. The alternative solution takes $O(\log(n) + \log(m))$ memory. Remember: memory usage can be a very big deal when it comes to scalability.
2. The simple solution is $O(n + m)$ time and the alternative solution has a worst case time of $O(nm)$. However, the worst case time can be deceiving; we need to look deeper than that.
3. A slightly tighter bound on the runtime, as explained earlier, is $O(n + km)$, where k is the number of occurrences of `T2`'s root in `T1`. Let's suppose the node data for `T1` and `T2` were random numbers picked between 0 and p . The value of k would be approximately $\frac{n}{p}$. Why? Because each of n nodes in `T1` has a $\frac{1}{p}$ chance of equaling the root, so approximately $\frac{n}{p}$ nodes in `T1` should equal `T2`'s root. So, let's say $p = 1000$, $n = 1000000$ and $m = 100$. We would do somewhere around 1,100,000 node checks ($1100000 = 1000000 + \frac{100 * 1000000}{1000}$).
4. More complex mathematics and assumptions could get us an even tighter bound. We assumed in #3 above that if we call `matchTree`, we would end up traversing all m nodes of `T2`. It's far more likely, though, that we will find a difference very early on in the tree and will then exit early.

In summary, the alternative approach is certainly more optimal in terms of space and is likely more optimal in terms of time as well. It all depends on what assumptions you make and whether you prioritize reducing

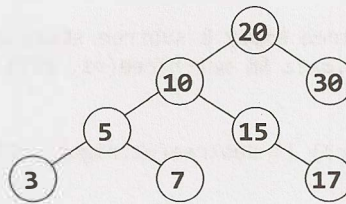
the average case runtime at the expense of the worst case runtime. This is an excellent point to make to your interviewer.

4.11 Random Node: You are implementing a binary search tree class from scratch, which, in addition to `insert`, `find`, and `delete`, has a method `getRandomNode()` which returns a random node from the tree. All nodes should be equally likely to be chosen. Design and implement an algorithm for `getRandomNode`, and explain how you would implement the rest of the methods.

pg 111

SOLUTION

Let's draw an example.



We're going to explore many solutions until we get to an optimal one that works.

One thing we should realize here is that the question was phrased in a very interesting way. The interviewer did not simply say, "Design an algorithm to return a random node from a binary tree." We were told that this is a class that we're building from scratch. There is a reason the question was phrased that way. We probably need access to some part of the internals of the data structure.

Option #1 [Slow & Working]

One solution is to copy all the nodes to an array and return a random element in the array. This solution will take $O(N)$ time and $O(N)$ space, where N is the number of nodes in the tree.

We can guess our interviewer is probably looking for something more optimal, since this is a little too straightforward (and should make us wonder why the interviewer gave us a binary tree, since we don't need that information).

We should keep in mind as we develop this solution that we probably need to know something about the internals of the tree. Otherwise, the question probably wouldn't specify that we're developing the tree class from scratch.

Option #2 [Slow & Working]

Returning to our original solution of copying the nodes to an array, we can explore a solution where we maintain an array at all times that lists all the nodes in the tree. The problem is that we'll need to remove nodes from this array as we delete them from the tree, and that will take $O(N)$ time.

Option #3 [Slow & Working]

We could label all the nodes with an index from 1 to N and label them in binary search tree order (that is, according to its inorder traversal). Then, when we call `getRandomNode`, we generate a random index between 1 and N . If we apply the label correctly, we can use a binary search tree search to find this index.

However, this leads to a similar issue as earlier solutions. When we insert a node or delete a node, all of the indices might need to be updated. This can take $O(N)$ time.

Option #4 [Fast & Not Working]

What if we knew the depth of the tree? (Since we're building our own class, we can ensure that we know this. It's an easy enough piece of data to track.)

We could pick a random depth, and then traverse left/right randomly until we go to that depth. This wouldn't actually ensure that all nodes are equally likely to be chosen though.

First, the tree doesn't necessarily have an equal number of nodes at each level. This means that nodes on levels with fewer nodes might be more likely to be chosen than nodes on a level with more nodes.

Second, the random path we take might end up terminating before we get to the desired level. Then what? We could just return the last node we find, but that would mean unequal probabilities at each node.

Option #5 [Fast & Not Working]

We could try just a simple approach: traverse randomly down the tree. At each node:

- With $\frac{1}{3}$ odds, we return the current node.
- With $\frac{1}{3}$ odds, we traverse left.
- With $\frac{1}{3}$ odds, we traverse right.

This solution, like some of the others, does not distribute the probabilities evenly across the nodes. The root has a $\frac{1}{3}$ probability of being selected—the same as all the nodes in the left put together.

Option #6 [Fast & Working]

Rather than just continuing to brainstorm new solutions, let's see if we can fix some of the issues in the previous solutions. To do so, we must diagnose—deeply—the root problem in a solution.

Let's look at Option #5. It fails because the probabilities aren't evenly distributed across the options. Can we fix that while keeping the basic algorithm the same?

We can start with the root. With what probability should we return the root? Since we have N nodes, we must return the root node with $\frac{1}{N}$ probability. (In fact, we must return each node with $\frac{1}{N}$ probability. After all, we have N nodes and each must have equal probability. The total must be 1 (100%), therefore each must have $\frac{1}{N}$ probability.)

We've resolved the issue with the root. Now what about the rest of the problem? With what probability should we traverse left versus right? It's not 50/50. Even in a balanced tree, the number of nodes on each side might not be equal. If we have more nodes on the left than the right, then we need to go left more often.

One way to think about it is that the odds of picking something—anything—from the left must be the sum of each individual probability. Since each node must have probability $\frac{1}{N}$, the odds of picking something from the left must have probability $\text{LEFT_SIZE} * \frac{1}{N}$. This should therefore be the odds of going left.

Likewise, the odds of going right should be $\text{RIGHT_SIZE} * \frac{1}{N}$.

This means that each node must know the size of the nodes on the left and the size of the nodes on the right. Fortunately, our interviewer has told us that we're building this tree class from scratch. It's easy to keep track of this size information on inserts and deletes. We can just store a `size` variable in each node. Increment `size` on inserts and decrement it on deletes.